



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
INGENIERÍA INFORMÁTICA
LENGUAJES Y COMPILADORES

TEMA 4: ANÁLISIS LÉXICO

DOCENTE:
FELIX MARQUEZ

GRUPO NABLA:

DANIEL VILLALBA SANTAMARIA
ISABELLA RODRIGUEZ
ANDRES GOMEZ
LUIS MILLAN

CIUDAD GUAYANA, JULIO 2026

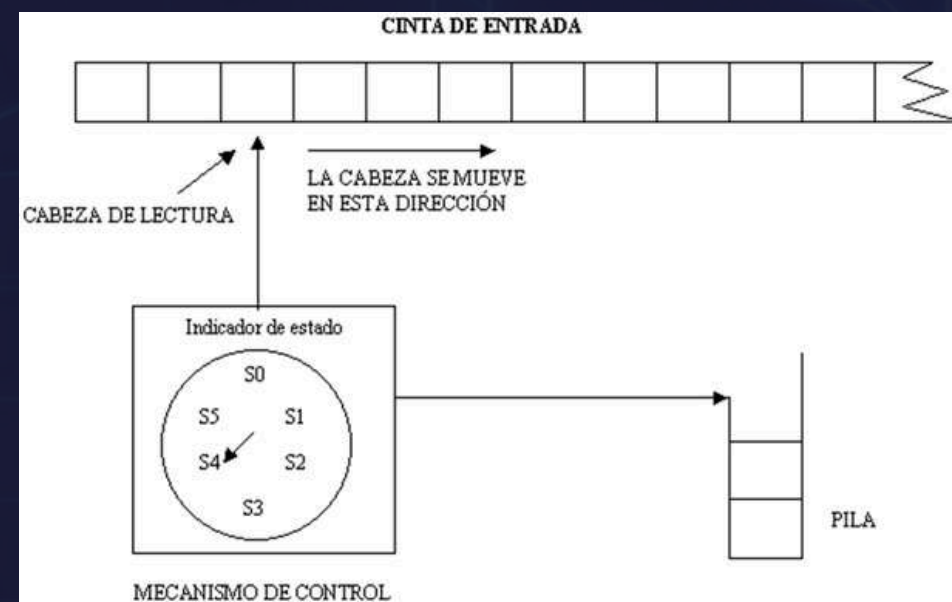


LENGUAJES Y COMPILADORES

¿QUÉ ES UN AUTÓMATA DE PILA?

Es una máquina abstracta de estados que extiende las capacidades de los autómatas finitos tradicionales al incorporar una memoria auxiliar externa

- Memoria de tipo LIFO (Last In, First Out).
- Capacidad de almacenamiento ilimitada.
- Acceso restringido únicamente al tope de la pila.





LENGUAJES Y COMPILADORES

DEFINICIÓN FORMAL (LA 7-TUPLA)

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Donde:

Q: Estados internos.
Sigma: Alfabeto de entrada.
Gamma: Alfabeto de pila.
delta: Función de transición.

q₀: Estado inicial.
Z₀: Fondo de pila.
F: Estados finales.



LENGUAJES Y COMPILADORES

RECONOCIMIENTO DE LENGUAJE NO REGULAR

Etapas del Proceso	Flujo en la Cinta de Lectura	Contenido de la Pila	Operación Ejecutada
Inicio del proceso	a a b b	[Z0]	Inicialización (Marcador de
Lectura carácter 1	(a) a b b	[A, Z0]	Apilar (<i>Push</i> de A)
Lectura carácter 2	a (a) b b	[A, A, Z0]	Apilar (<i>Push</i> de A)
Lectura carácter 3	a a (b) b	[A, Z0]	Desapilar (<i>Pop</i> de A)
Lectura carácter 4	a a b (b)	[Z0]	Desapilar (<i>Pop</i> de A)
Fin de la cadena	a a b b (ϵ)	[Z0]	Aceptación por Pila Vacía / Estado Final



LENGUAJES Y COMPILADORES

APLICACIÓN EN COMPILADORES

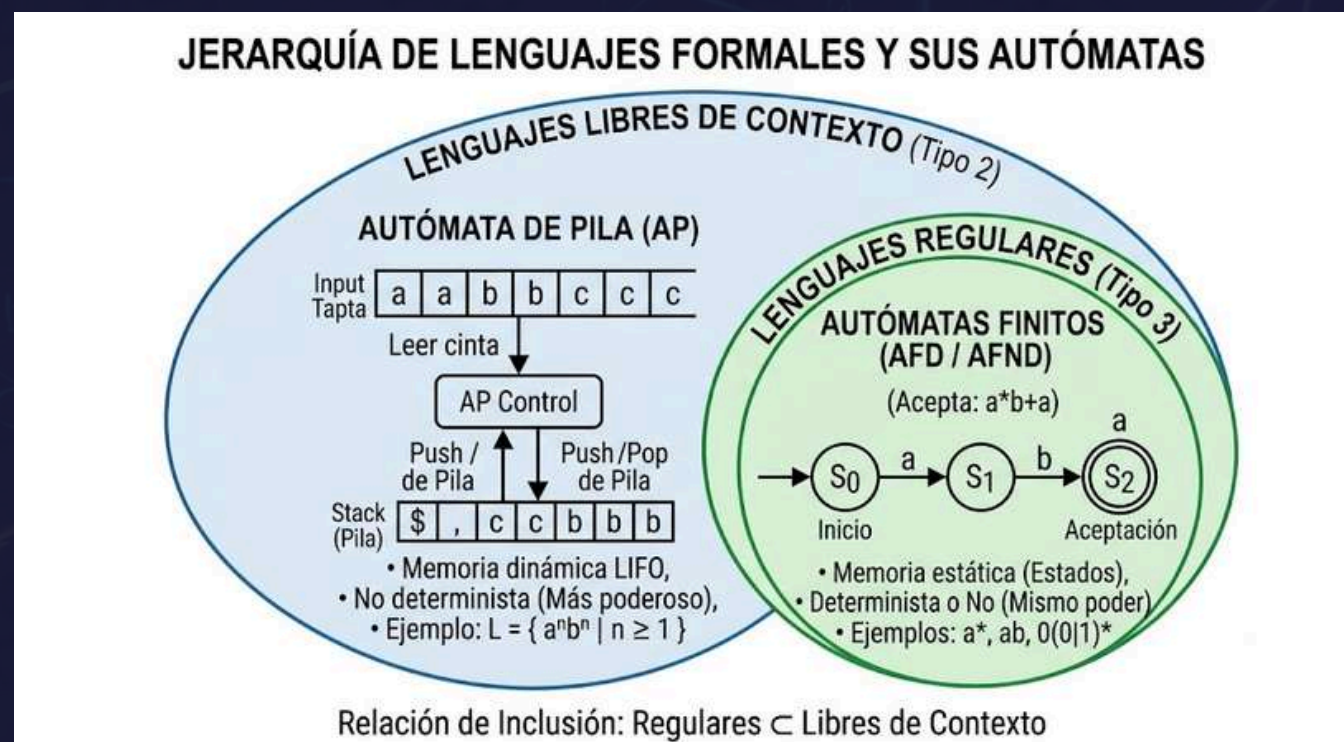
Etapas del Proceso	Flujo en la Cinta de Lectura	Contenido de la Pila	Operación Ejecutada
Inicio del proceso	(())	[Z0]	Inicialización (Marcador de fondo)
Lectura apertura 1	(())	[(, Z0]	Apilar (<i>Push</i> de apertura)
Lectura apertura 2	(())	[(, (, Z0]	Apilar (<i>Push</i> de apertura)
Lectura cierre 1	(())	[(, Z0]	Desapilar (<i>Pop</i> por coincidencia)
Lectura cierre 2	(())(ε)	[Z0]	Desapilar (<i>Pop</i>) y Aceptación



LENGUAJES Y COMPILADORES

IMPORTANCIA Y PODER DEL AP VS. AFD / AFND

- AFD / AFND:
Memoria estática.
Lenguajes Regulares.
- Autómatas de Pila:
Memoria dinámica.
Libres de Contexto





LENGUAJES Y COMPILADORES

CONSTRUCCION DE UN LEXER PARA VERIFICACION DE DOCKERFILES MEDIANTE REGEX



docker®



DANIEL VILLALBA SANTAMARIA



OBJETIVO DEL ANALIZADOR LEXICO

Tema 4 > Actividad 2 >  dockerfile_1.txt

```
1  # Ejemplo de Dockerfile
2
3  FROM ubuntu:20.04
4  WORKDIR /app
5  COPY . /app
6  CMD "servidor.py"
```




LENGUAJES Y COMPILADORES

ELEMENTOS IGNORABLES

PALABRAS RESERVADAS

ESTRUCTURAS CERRADAS

VALORES GENERICOS

MANEJO DE ERRORES

DICCIONARIO DE TOKENS Y EXPRESIONES REGULARES PARA DOCKERFILES

Nombre del Token	Expresión Regular (Regex)	Descripción / Lexema Esperado
TK_COMMENT	#.*	Identifica comentarios para ser ignorados por el lexer.
TK_SPACE	[\t]+	Identifica espacios en blanco y tabulaciones.
TK_NEWLINE	\n	Detecta los saltos de línea.
TK_FROM	\bFROM\b	Instrucción base para definir la imagen.
TK_RUN	\bRUN\b	Instrucción para ejecutar comandos de terminal.
TK_WORKDIR	\bWORKDIR\b	Instrucción para establecer el directorio de trabajo.
TK_COPY	\bCOPY\b	Instrucción para copiar archivos al contenedor.
TK_CMD	\bCMD\b	Instrucción que define el comando por defecto al iniciar.
TK_STRING	"[^"]*"	Captura cadenas de texto encerradas entre comillas dobles.
TK_ARGUMENT	[a-zA-Z0-9_./:\-]+	Captura argumentos generales (rutas, versiones, nombres).
TK_MISMATCH	.	Atrapa cualquier carácter no reconocido para detonar un error léxico.



LENGUAJES Y COMPILADORES

DISEÑO Y AUTOMATIZACIÓN DE ANALIZADORES LÉXICOS Y MODELOS DE CÓMPUTO.

LUIS MILLAN



LENGUAJES Y COMPILADORES

INTRODUCCIÓN A L_{RUST} Y LA SINTAXIS DEFINIDA

CATEGORÍA	EXPRESIÓN REGULAR	EJEMPLO DE LEXEMA
PALABRAS CLAVE	"FN", "LET", "MUT", "STRUCT"	FN, LET, MUT
OPERADORES	"->", "==", "+", "-"	->, ==, +
TIPOS DE DATOS	"I32", "F32", "BOOL"	I32, BOOL



LENGUAJES Y COMPILADORES

ARQUITECTURA DEL METACOMPILADOR FLEX

REPRESENTACIÓN DE LA ESTRUCTURA

DEFINICIONES



REGLAS



CODIGO C++

```
lex.yy.c
41  "/*" { token_line = line_num; token_col = col_num; update_location(yytext); }
42  [ \t\r]+ { token_line = line_num; token_col = col_num; update_location(yytext); }
43  \n { line_num++; col_num = 1; }
44
45  "fn" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_FN; }
46  "let" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_LET; }
47  "mut" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_MUT; }
48  "if" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_IF; }
49  "else" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_ELSE; }
50  "return" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_RETURN; }
51  "true" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_TRUE; }
52  "false" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_FALSE; }
53
54  "i32" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_I32; }
55  "f32" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_F32; }
56  "bool" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_BOOL; }
57
58  ">" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_ARROW; }
59  "==" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_EQ; }
60  "!=" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_NE; }
61  "<" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_LT; }
62  ">" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_GT; }
63  "=" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_ASSIGN; }
64  "+" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_PLUS; }
65  "-" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_MINUS; }
66  "*" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_MUL; }
67  "/" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_DIV; }
68
69  "{" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_LBRACE; }
70  "}" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_RBRACE; }
71  "(" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_LPAREN; }
72  ")" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_RPAREN; }
73  ":" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_COLON; }
74  ";" { token_line = line_num; token_col = col_num; update_location(yytext); return TK_SEMI; }
75  "," { token_line = line_num; token_col = col_num; update_location(yytext); return TK_COMMA; }
```




LENGUAJES Y COMPILADORES

ROBUSTEZ DEL SISTEMA: GESTIÓN DE ERRORES LÉXICOS



EL LEXER NO SE DETIENE AL HALLAR UN CARÁCTER INVÁLIDO

TOKEN	LEXEMA	LINEA	COLUMNA
TK_FN	fn	1	1
TK_IDENT	main	1	4
TK_LPAREN	(	1	8
TK_RPAREN	)	1	9

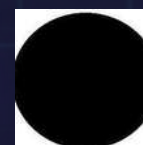
ERROR LEXICO: caracter invalido '\$' en la linea 14, columna 14

TK_SEMI	;	14	15
TK_LET	let	15	5
TK_IDENT	b	15	9
TK_ASSIGN	=	15	11

ERROR LEXICO: caracter invalido '@' en la linea 15, columna 13

```
void update_location(const char* text) {  
    for (int i = 0; text[i] != '\0'; ++i) {  
        if (text[i] == '\n') {  
            line_num++;  
            col_num = 1;  
        } else if (text[i] != '\r') {  
            col_num++;  
        }  
    }  
}
```

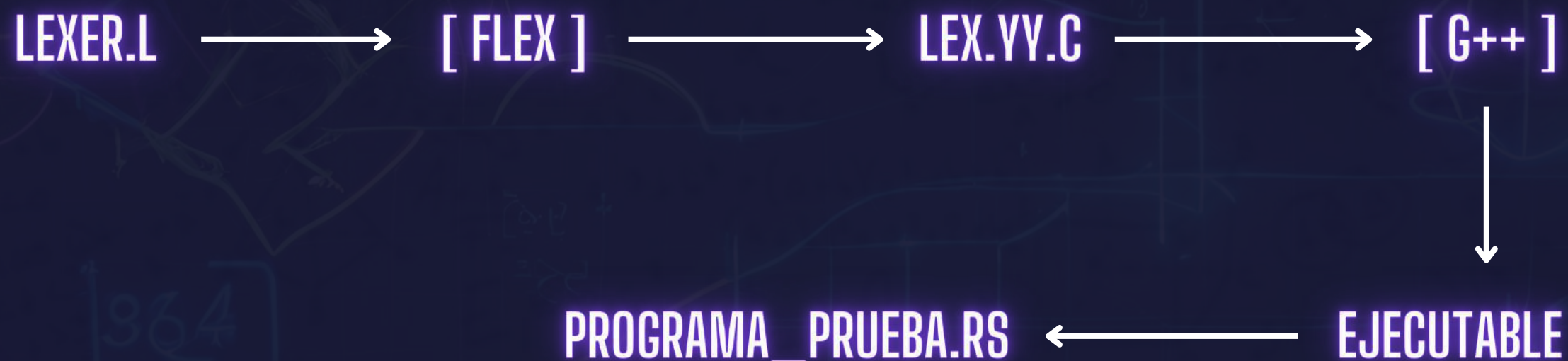
REGLA COMODÍN:





LENGUAJES Y COMPILADORES

FLUJO DE EJECUCIÓN Y PRUEBAS





LENGUAJES Y COMPILADORES

¿POR QUÉ FLEX EN CIBERSEGURIDAD?

- DESAFÍO DE LA DETECCIÓN
- SINTAXIS BASADA EN FIRMAS
- EL ROL DEL LEXER
- LA VENTAJA DE FLEX





LENGUAJES Y COMPILADORES

ANATOMÍA DE UNA REGLA Y SU TOKENIZACIÓN

ALERT TCP \$EXTERNAL_NET ANY -> \$HOME_NET 80 (MSG:"INTRUSIÓN WEB"; SID:100001;)

Lexema (Fragmento)	Token Identificado	Propósito / Significado
alert	TK_ACTION	Acción a tomar cuando se cumple la regla
tcp	TK_PROTOCOL	Protocolo de red a filtrar
\$EXTERNAL_NET	TK_VAR	Variable de red de origen (Regex: \\${A-Z}_+)
any / 80	TK_PORT	Puertos de origen/destino de la conexión
->	TK_DIRECTION	Dirección del flujo de tráfico de red
msg / sid	TK_OPTION_KEY	Palabras clave de las opciones internas de Snort
"Intrusión Web"	TK_OPTION_VALUE	Valor asignado a la opción de alerta (String)



LENGUAJES Y COMPILADORES

IMPLEMENTACIÓN CONCEPTUAL EN FLEX

```
%%  
"alert"           { return TK_ACTION; }  
"tcp"|"udp"|"any" { return TK_PROTOCOL; }  
"->"             { return TK_DIRECTION; }  
$[A-Z_]+          { return TK_VAR; }  
[0-9]+            { return TK_PORT; }  
\"[^\"]*\"          { return TK_OPTION_VALUE; } // Cadenas con comillas  
[a-zA-Z0-9_-]+    { return TK_IDENTIFIER; }  
.  
%%  
/* Ignorar o gestionar error léxico */
```




LENGUAJES Y COMPILADORES

¡GRACIAS POR SU ATENCIÓN!